Speed limit

1. Course Content

Note: This program runs on a sandbox map; you will need to adjust the program for your personal map.

1. Learn the actions the car takes after recognizing speed limit
2. Learn the newly emerging key source code

2. Road sign detection

Road sign detection source code path:

```
/home/jetson/yahboomcar_auto/src/auto_driver/scripts/auto_drive.py
```

3. Program Startup

3.1 Startup Command

- Parameter configuration

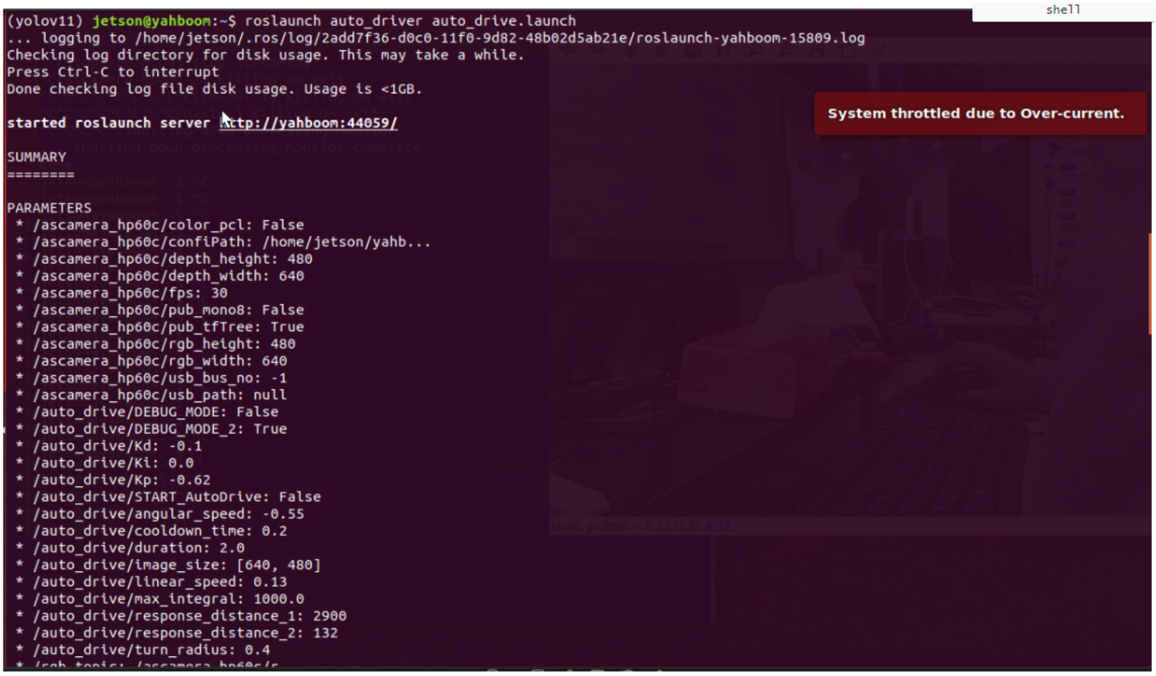
Parameter path: `/home/jetson/yahboomcar_auto/src/auto_driver/config/auto_drive_node.yaml`

```yaml
turn_radius: 0.4
linear_speed: 0.13
angular_speed: -0.55
duration: 2.0
response_distance_1: 2900
response_distance_2: 132
cooldown_time: 0.2
DEBUG_MODE: False
DEBUG_MODE_2: True
START_AutoDrive: True
Kp: -0.62
Ki: 0.0
Kd: -0.1
max_integral: 1000.0
image_size: [640, 480]
```

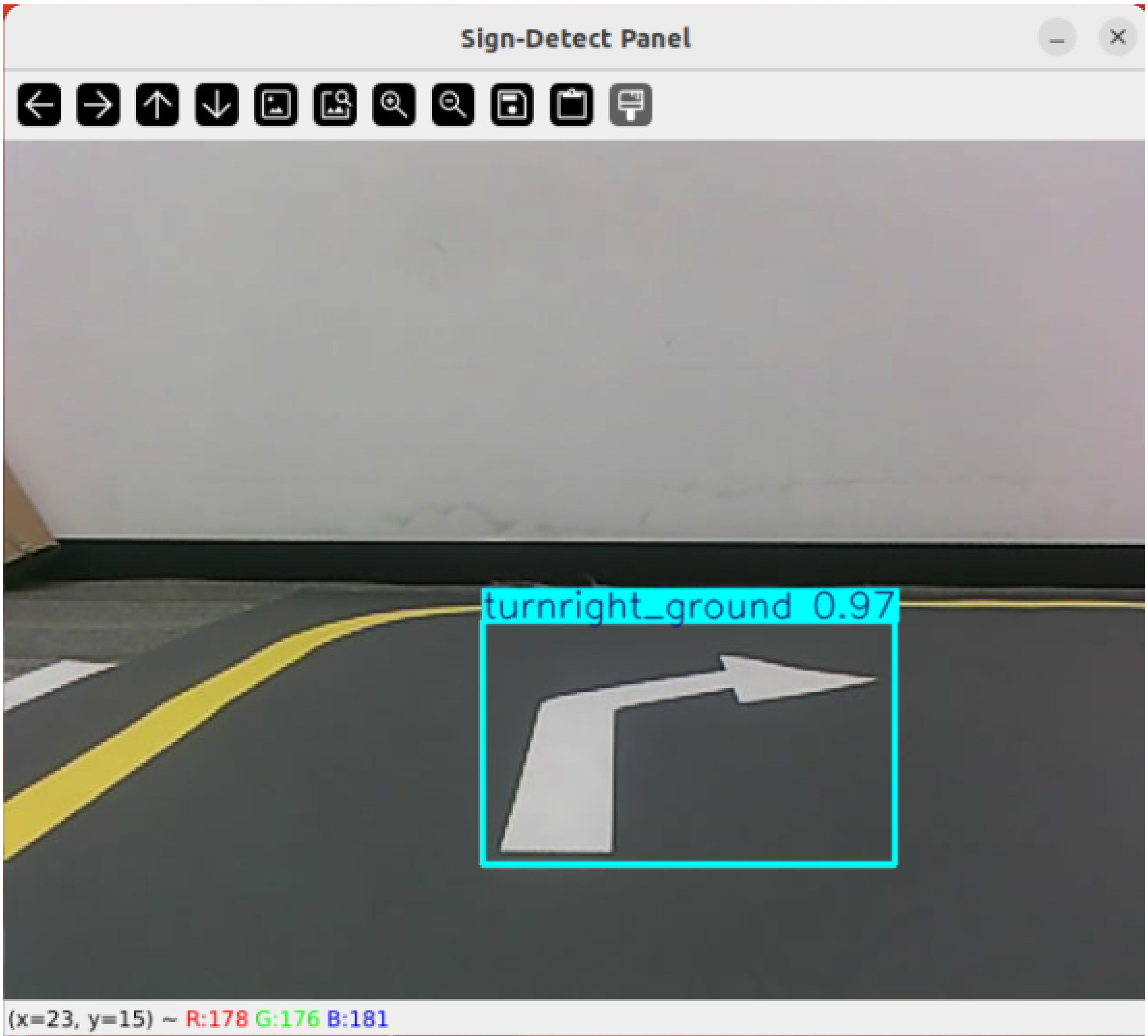
After modifying parameters, restart the launch file to use the modified parameters

- Start camera + chassis + autonomous driving node (need to run in virtual environment terminal)

```
roslaunch auto_driver auto_drive.launch
```



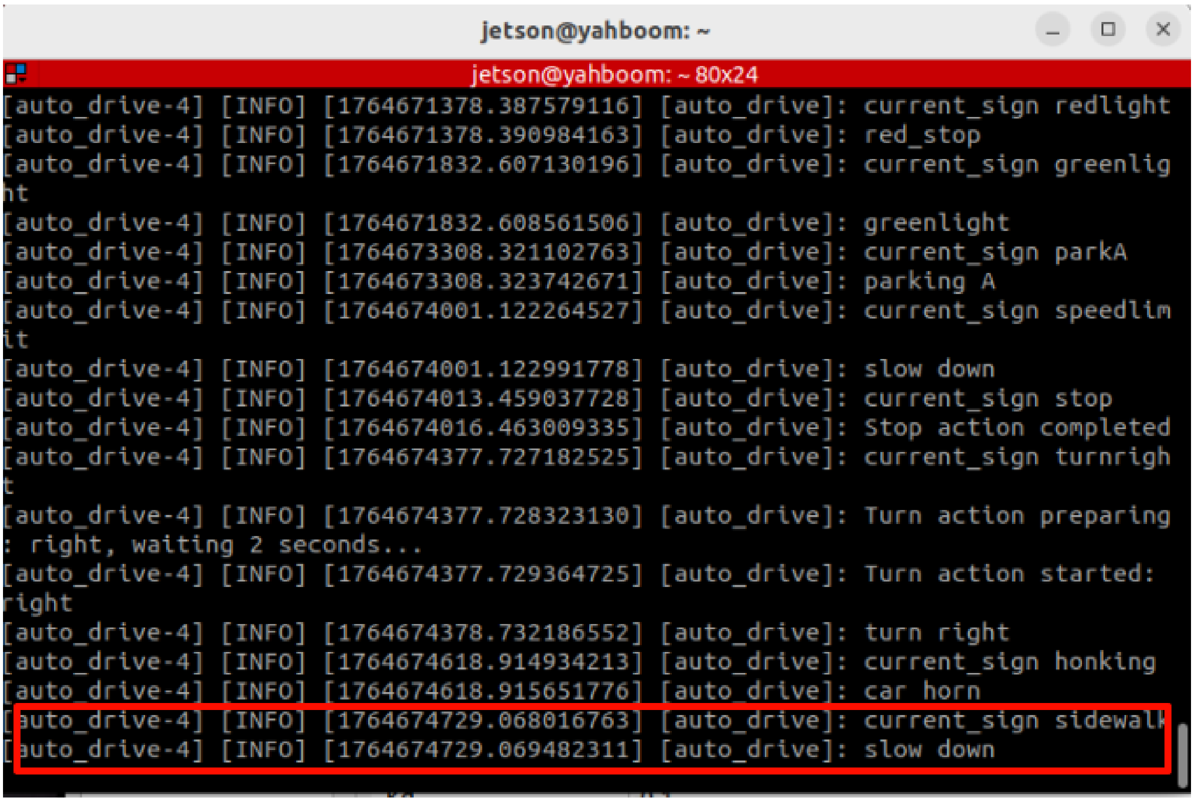
After successful startup, the display will show but the car will not move, for testing road sign recognition. (If you want the car to move, change the parameter START_AutoDrive in auto_drive_node.yaml to True)



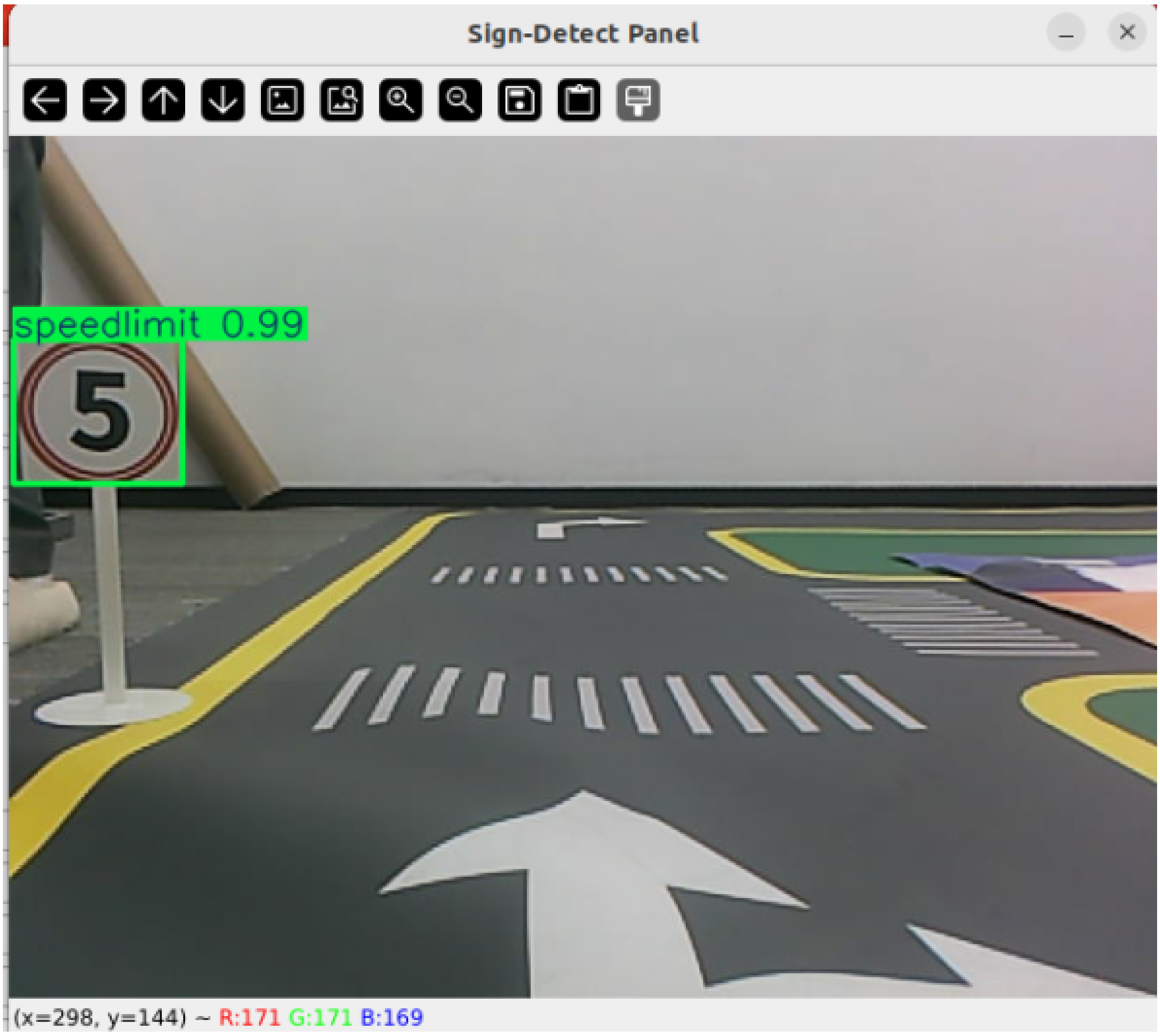
After the program runs, the YOLO recognition screen will be displayed. The car will run centered on the lane lines and perform corresponding actions based on the road signs recognized during driving.

3.2 Car movement after recognizing speed limit

When the program recognizes speed limit, the program will decelerate for 3 seconds then return to normal



The car will decelerate for 3 seconds while driving



The fixed point parking handling function is in the auto_drive.py file. Below is the function that is entered after recognizing speed limit. Users can modify the car's movement after recognizing the road sign themselves.

```
def slow_down(self):
    rospy.loginfo('Slow down')
    self.flag = True  #Indicates temporary lane line driving
    self._set_current_speed(linear_x=0.08)
    time.sleep(3.0)
    self.flag = False
```

4. Source Code Analysis

4.1、auto_drive.py

YOLO signpost message reception:

```
results = self.sign_detect.get_infer_result(frame, True)
annotated_frame = frame.copy()
```

Multiple filtering mechanisms: mainly to prevent continuously recognizing road signs and triggering actions; the action will only be unlocked when different road signs are recognized.

```
 # If a sign that requires cooldown is detected, start cooldown
if detected_sign_requires_cooldown:
    self.sign_cooldown = True
    self.last_sign_time = current_time
    rospy.loginfo(f"Sign detected, starting {self.cooldown_duration}s cooldown")
 # Check if the repeated sign should be ignored
should_ignore = False
if obj.class_name == self.last_sign:
        should_ignore = True
```

Event queue processing

```
def _event_handling_worker(self):
        '''Event handling worker thread'''
        # Sign handling mapping table
        sign_handlers = {
            "straight": self.go_straight,
            "turnright": lambda: self.turn('right'),
            "turnright_ground": self.do_nothing,
            "honking": self.car_horn,
            "sidewalk": self.slow_down,
            "sidewalk_ground": self.do_nothing,
            "speedlimit": self.slow_down,
            "stop": self.stop,
            "school": self.school,
            "parkA": self.parking_A,
            "parkB": self.parking_A,
            "greenlight": self.go_straight,
            "redlight": self.red_stop,
            "yellowlight": self.stop,
            "out_park": self.out_parking
        }

        # Sign enable index mapping
        sign_enable_map = {
            "straight": 10, "turnright": 11, "honking": 1, "sidewalk": 6,
            "sidewalk_ground": 7, "speedlimit": 8, "stop": 9, "school": 5,
            "parkA": 2, "parkB": 3, "greenlight": 0, "redlight": 4,
"yellowlight": 13
        }
```

Specific action implementation

```
def slow_down(self):
    rospy.loginfo('Slow down')
    self.flag = True
    self._set_current_speed(linear_x=0.08)
    time.sleep(3.0)
    self.flag = False
```